Cake Bakery

PCLT Project



Authors:

Daniel Eugénio 59797

Renato Viola 60665

NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

In this report, we discuss the various design choices made during the implementation of a simulation of a cake bakery with concurrent workers, time passing by, and concurrent customer arrival and cake production.

Entity Design Modeling

We began the project by modeling every entity described in the proposal, as well as some additional structures we found useful during the development of the system. This is a high-level explanation of what each component does. Technical solutions (specifically the concurrency management) are further discussed in the section Concurrency Control in Critical Resources.

CustomerQueues

This structure includes both client queues, regular and priority. This is so that both queues share the same conditional variable. The *"condvar"* is used to signal to one single worker that a new customer arrived, after a period where the bakery was empty. We did this to save resources when no client is in the store waiting to be served, as the worker threads are paused without consuming CPU cycles needlessly, attempting to fetch a client in a probably still empty queue. The conditional variable is also used to signal to all workers that the day is finished. This was done after we noticed that employees could try to pick up a new customer while the store was open, but the store would then close after entering that function, leaving a thread waiting for a new customer to arrive if there was nobody waiting to be serviced yet. Since no customers arrive after closing, the worker thread would just hang there indefinitely.

DailyStats

DailyStats is a structure that gathers information such as portions of cake sold, leftovers and how many portions each worker sold on that particular day. It serves no other real purpose than to organize this information for the Bakery structure, so it can properly display it once the simulation's over.

Bakery

We decided that for our implementation, the bakery should act as the main component which runs the workers threads, but also as the statistics holder, so its only attribute is a vector of DailyStats (as mentioned previously).

The 'operate' function simulates the bakery being open for a single day, it spawns the worker threads with all the information they require to function and then gathers the results of each worker for that day. All the other methods the Bakery implements are stats-oriented, where the first computes the statistics for one single day and displays them, and the second one takes the previously calculated values and aggregates them to generate the stats

across all days. This information includes the amount of portions sold, amount of leftover pieces and the best worker. The best worker is calculated not by the most portions sold in a single day, but by the average of portions of cake sold across all days. This was done to promote consistency over individual quantity.

Client

Clients carry only 3 properties: an identification for logging purposes, their order in portions, generated randomly when they are created and if they are a priority client, also computed at their creation. There is a 20% chance clients are prioritary. They serve no other function at the system other than being inserted into a queue.

Worker

Workers carry most of the functionality of the system. As attributes, each worker has an ID, a flag to indicate if they prioritize the priority clients or not and the number of portions they've sold at a given day.

Worker implementation includes two operations: fetching a new client and the service function. The first one gets a new client from the queue when one is already waiting. If no clients are in the system, the worker thread pauses execution until a notice is issued that a customer is waiting. The service function acts as the brain of the whole system, defining the expected behavior from workers. It operates while the bakery is open, fetching clients and getting cakes from the queue disguised like a stack, "serving" the current client (logically speaking, no actual transaction happens) and finally updating their own stats.

System flow

The system starts by "creating" the bakery and getting the "duration" in days for the simulation as input from the user. For each day, we instantiate the new empty client queues, a cake queues that start with 20 pre-baked cakes, as well as two flow control flags that represent the passage of time in a day, 'is_open' indicates if the bakery is open and a 'is_morning' indicates the if it's either the morning or the afternoon (this is important because it dictates the flow of the cake production).

We believe the bakery shouldn't concern itself with implementing the logic of the flow of time or the cake production and the arrival of customers, these are "external" things happening in the background and the bakery should simply be able to deal with them, or rather let its workers deal with these factors. So each of these elements is actually spawned as its own thread and happens parallel to each other. Probably the most interesting one is the one that handles the passage of time along a day of work, as this one dictates the "lifetime" of all the others, including the bakery itself. It simulates the morning/afternoon cycle and changes the value of the 'is_open' and 'is_morning' flag accordingly. Customer arrival basically consists of clients arriving in intervals with a normal distribution of mean 1 and standard deviation of 0.1. Cakes are consistently produced in the morning and production stops once it's afternoon, but 50 more cakes are available by then. It goes without saying that there is only cake production, customer arrival and bakery operation if it's actually the

time to be open. Before finishing off for the day, we print the day's statistics and join the threads.

Concurrency Control in Critical Resources

‘is_open’ and ‘is_morning’ flags

RwLocks were used to lock the “is_open” and “is_morning” flags, which represent the bakery’s operation status and time of day. This was done since only in the day time logic function were these values ever written to, and that only happens once in a day. All the other threads only read these values so they shouldn’t have to wait for each other.

Customer Queues

The customer queues are accessed concurrently by all workers when attempting to fetch a client, making it a critical resource in need of protection. To implement said protection, we opted by guarding these queues with a mutex alongside a conditional variable, so that only one worker may access the queue at a time to retrieve a client but also so that it may sleep while waiting for a client to arrive (thus not wasting CPU resources). Once a customer arrives, one worker that was “sleeping” is notified, and may now get to work. Once it’s closing time, every worker thread is notified so that they may know that the day is over and not to expect any more customers to attend to.

Cake queue

Much like the customer queues, the cake queue is concurrently accessed by the workers during service. To ensure the security of operations, we once again locked the structure using a mutex, especially since nearly all accesses to it are write operations (a read-only operation only occurs when calculating the portions left for a day, but this is after the day is already and no threads are competing for this structure). No use of a conditional variable this time, since there is no waiting for cake availability. A customer should not have to be kept waiting, the worker just accepts that it can’t complete this customer’s demand and tries to attend to the next one, hopefully having enough cake portions to serve it.

Deadlocks and Starvation

Wherever we could, we avoided nesting locks on different resources and tried to keep the locks’ scopes to a minimum. With the use of Condvar we also intended to avoid workers busy waiting for a customer retrieval and thus also avoided acquiring locks on meaningless iterations (that wouldn’t retrieve a customer regardless). When the bakery closes, all workers are notified of it and know that they shouldn’t try to attend to any other customer. All workers have a fair shot at retrieving a client, since any of them that is waiting on an arrival is notified when it happens. Access to cakes is fair as well.

Additional Comments

Color-coded logs for easier debugging

When testing our implementation by running the program, the console would obviously get cluttered with information regarding several different events. This made it quite difficult to keep track of what was going on and keep track of the sequence of events taking place, so we decided to color code different logs by worker. Each worker is assigned a color and their actions are always printed in the same color for the whole execution. This eases the perception of who does what.

Custom simulation duration

In our system, the user chooses the number of days the simulation runs for. We initially thought of making a fixed 4-day work week (since workers work 10 hour days). This would require the user to sit for at least 4 minutes looking at a terminal. Our solution is more flexible and allows the user to see what it really wants to see, even if it's a single day.

Queues in our system are VecDeques

We opted by implementing the queues in our system as VecDeques, for the convenient and powerful functionalities it provides, our main deciding factor was the freedom in retrieving and inserting elements both at the head and at the tail of the data structure.